GRAFANA · POSTGRESQL · AZURE DATA FACTORY · PYTHON · FINOPS

Grafana AI Cost Observability

Turning Gen AI, GitHub Copilot and Copilot Studio consumption into a single, cost-centre-level view of AI spend — built on the same ingestion and chargeback pattern already proven for Azure Cost Management.

Objective	Give every segment, tower and cost centre an explainable monthly view of AI spend across three independently billed Microsoft AI products.
Stack	Azure Data Factory (orchestration) → Python transformation modules → PostgreSQL (serving) → Grafana (four-tab dashboard).
Scope	Gen AI / Azure OpenAI & Foundry, GitHub Copilot (user-level), Copilot Studio (credit-based).
My role	Data model, Python transformation layer, ADF pipeline design, PostgreSQL schema and all Grafana panels.

The problem

AI spend arrived from three different places, in three different shapes, on three different cadences — and none of them agreed on what a “cost centre” was.

Source	Grain it arrives at	Why it resists consolidation
Gen AI / Azure OpenAI	Azure meter × resource × day	Priced per meter category and sub-category; subscription tags are inconsistent, and Foundry Models / Foundry Tools only split out from Cognitive Services part-way through the year.
GitHub Copilot	Licensed user × month	A seat licence, not a consumption meter. Must be joined to HR data to reach a cost centre, and seat counts churn monthly.
Copilot Studio	Environment × consumed credits	Billed in credits, not currency, and the tenant-to-environment capacity change mid-year broke the comparability of the monthly series.

- **No shared key.** Meter-level Azure data, per-user licence data and per-environment credit data had no common identifier. Each needed its own route to a cost centre.
- **Different units.** Credits had to be converted to currency before anything could be summed.
- **Unstable history.** Microsoft re-categorised meters and moved Copilot Studio capacity mid-year, so a naive month-over-month comparison reported a cost collapse that never happened.
- **Two audiences.** Finance needed segment and cost-centre totals that reconcile to the invoice; engineering needed model-, environment- and user-level detail to act on.

> **Design principle carried over from the Azure Cost Management build**
>
> Allocate once, in the transformation layer, and make the dashboard a thin read over an already-reconciled table. Any logic pushed into Grafana panels becomes logic that cannot be tested, reconciled or reused.

ARCHITECTURE

Pipeline

The pipeline reuses the Azure Cost Management pattern end to end: ADF orchestrates, Python owns every transformation, PostgreSQL serves, Grafana renders.

Stage	What happens
1 · Ingest	ADF pipelines land raw exports on a schedule — Azure cost exports for Gen AI, the Copilot licence extract, and the Copilot Studio credit export — each into its own staging area with the run date attached.
2 · Transform (Python)	Per-source Python modules normalise currency and dates, map meters to product and meter sub-category, convert Copilot Studio credits to charges, and resolve every record to a cost centre through the ownership hierarchy. Tax is computed separately so it can be toggled on or off at query time.
3 · Allocate	Resolved records are folded into the shared segment / sub-segment / tower / function hierarchy so AI spend aggregates the same way as the rest of the cloud estate.
4 · Serve (PostgreSQL)	Monthly fact tables plus narrow dimension tables, indexed on the filter columns the dashboard exposes (month, segment, sub-segment, tower, function, cost centre).
5 · Visualise (Grafana)	One dashboard, four tabs. Template variables drive every panel, so a single filter set applies across all three products simultaneously.

Why Grafana rather than a BI tool

- The team already operated Grafana for platform telemetry — no new tool, licence or access model.
- Template variables give one filter bar across four tabs without duplicating queries per panel.
- PostgreSQL is queried directly, so there is no extra semantic layer to keep in sync with the warehouse.
- Dashboards version as JSON and review like code.

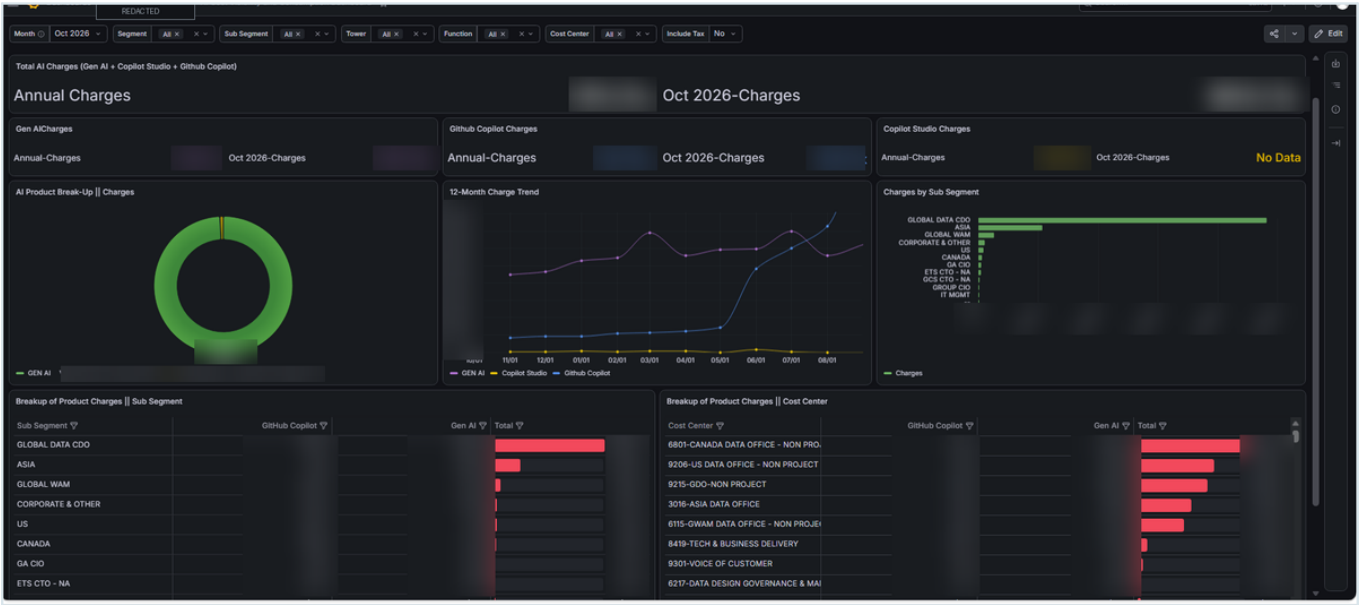
Cost-centre resolution

Resolution runs as an ordered waterfall — explicit finance override, then validated resource tag, then inventory and parent mappings, then history, then a regional default. Every record carries the rule that resolved it, so an unexpected allocation can always be traced back to the step that produced it.

THE DASHBOARD

Four tabs, one filter bar

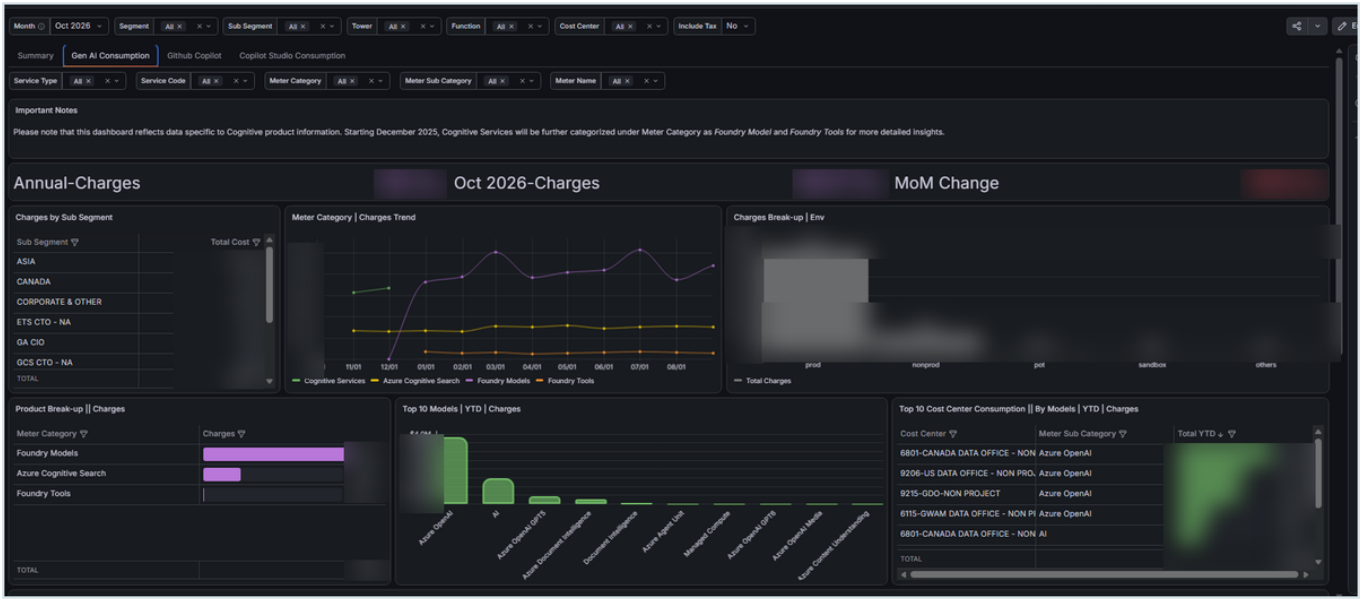
The Summary tab answers “what are we spending on AI in total, and who is spending it?” The three product tabs answer “why?”



Summary tab — total AI charges across Gen AI, GitHub Copilot and Copilot Studio, with product split, a 12-month trend, and breakdowns by sub-segment and cost centre. One filter bar (month, segment, sub-segment, tower, function, cost centre, include-tax) drives every tab.

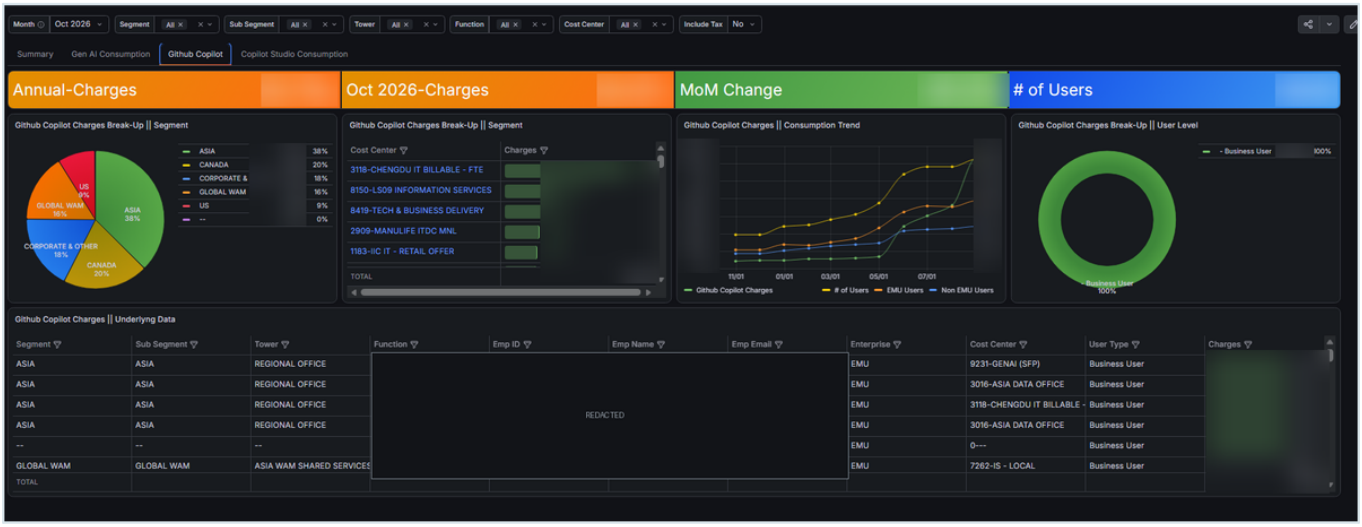
Design decisions worth noting

- **Annual and current-month side by side.** Every stat panel shows both, because the single most common question is “is this month unusual?”
- **Include Tax is a toggle, not a separate dashboard.** Finance reconciles with tax; engineering compares without it. Pre-computing both in the fact table makes it a filter.
- **Product split up front.** Gen AI dominates spend; surfacing that share immediately stops teams optimising the wrong product.



Gen AI Consumption tab — meter-category trend, environment split, top models by YTD charge, and top cost centres by model. The in-dashboard note documents the Dec-2025 re-categorisation of Cognitive Services into Foundry Models and Foundry Tools, so the trend break is explained where it is seen.

The environment split is the actionable panel here: non-production and sandbox spend on generative models is where unreviewed experimentation accumulates, and separating it from production spend turns a single large number into a conversation about which workloads actually need frontier models.



GitHub Copilot tab — seat charges by segment and cost centre, a consumption trend that overlays charges against total, EMU and non-EMU user counts, and a user-level detail table. Personal columns are redacted in this published version.

Copilot is a licence, not a meter

Because Copilot is billed per seat, cost per active user — not total cost — is the metric that matters. Plotting charges against user counts on a shared time axis makes the two separable: a rising line with flat seats is a price change, while flat charges with falling seats is unreclaimed licence waste. The user-level table exists so a cost-centre owner can identify dormant seats and reclaim them.



Copilot Studio tab — consumed credits and converted charges by sub-segment, cost centre and environment, with a production vs non-production monthly comparison. The disclaimer panel records the Microsoft capacity change that caused the November drop.

Documenting discontinuities inside the dashboard

Copilot Studio charges fall sharply from November because Microsoft moved capacity drawing from tenant level to environment level — a platform change, not reduced usage. The explanation lives in a panel directly above the chart. A trend break that is not explained where it is seen will be mis-read as a saving, and someone will claim credit for it.

OUTCOME

What it changed

Before	After
Three separate exports reconciled by hand in spreadsheets, monthly.	One scheduled pipeline; the dashboard is current without manual effort.
AI spend invisible below segment level.	Every charge resolves to a cost centre, with the resolution rule recorded.
Copilot Studio reported in credits, not currency.	Credits converted in the transformation layer; all three products sum in one number.
“Why did this change?” answered by email.	Meter re-categorisation and the capacity change documented in-panel, next to the trend.
Finance and engineering looked at different extracts.	One dashboard; the Include Tax toggle serves both reconciliation and comparison.

What I would do next

- **Unit economics.** Cost per active Copilot seat and cost per 1K tokens by model, so efficiency is trackable independently of volume.
- **Budget thresholds.** Grafana alerting on cost-centre spend against plan, rather than relying on someone opening the dashboard.
- **Anomaly detection.** Port the model already used for general cloud spend onto the AI fact tables — the Gen AI series is volatile enough to justify it.
- **Forecast overlay.** Project month-end spend from partial-month consumption, which is the question finance actually asks mid-month.

> **Note on this document**
>
> All screenshots are redacted for publication. Employee names, email addresses and employee IDs are permanently removed, and absolute financial values are obscured. Panel layout, chart structure and dashboard design are unchanged. Figures shown are illustrative and are not representative of any organisation's actual AI expenditure.